06. 고급파이썬프로그래밍 중간고사 — 필수 스펠링·사용법 완전 암기 노트

> **공부 대상**: 제공된 5개 노트북(`01_KNN_` → `05_ _ _`) + 교수님 중간고사 범위 안내 + 시험 정리 녹음본
> **시험**: 2026년 10월 21일(수) 수업 시간 / **범위**: 5장 선형회귀(단항·2차 다항회귀 포함)까지
> **문제 유형**: 괄호 채우기, 일부 코드 작성, 실행 결과·의미 파악 등 **코드 중심 단답형**. 교수님은 코드 전체를 처음부터 쓰는 문제는 내지 않는다고 설명함.
> **작성 원칙**: 노트북에서 실제로 사용된 철자·인수·코드·결과를 빠짐없이 대상으로 삼되, 암기를 돕기 위해 새로 만든 연습 예시는 `[ ]`로 표시. 실제 출제 문항이나 만점을 보장하는 문서가 아님.

목차

1. 시험에 필요한 영문 철자 전체 지도
2. `import`: 라이브러리·패키지·클래스 정확히 쓰기
3. 파이썬 기본 구문: 리스트·인덱싱·반복·출력·문자열
4. 입력·정답 데이터 구성과 NumPy 함수
5. 훈련·테스트 분할과 데이터 배열의 `shape`
6. 모든 모델의 공통 사용법: `fit`, `score`, `predict`
7. K-최근접 이웃 **분류**: 확률·이웃·이웃 수
8. **표준화(Z-score)**: `mean`, `std`, `axis`와 좌표
9. K-최근접 이웃 **회귀**: 예측·R^2·MAE·과대/과소적합
10. 단항 **선형회귀**: `LinearRegression`, `coef_`, `intercept_`
11. **2차 다항회귀**: 제곱 열·예측·곡선 그래프
12. Matplotlib 그래프 함수·옵션 한 글자씩 암기
13. 자료별로 구분해야 할 실제 출력·값
14. 틀리기 쉬운 철자·인수·문제 함정 총정리
15. 시험장에서 응용하는 완성 코드 패턴
16. **실전 빈칸·결과 예측·오류 찾기 100문제**
17. **100문제 정답과 해설**
18. 최종 암기 체크리스트

---

1. 시험에 필요한 영문 철자 전체 지도

한 번에 외울 때는 **어디서 가져오는가 → 어떤 모델을 만드는가 → 무슨 메서드를 호출하는가 → 무엇을 반환하는가** 순서로 외운다.

분류	정확한 철자	뜻·대표 사용법	암기 주의
별칭	`np`	`import numpy as np`	소문자 2글자
별칭	`plt`	`import matplotlib.pyplot as plt`	소문자 3글자
패키지	`sklearn`	사이킷런	`sklrean` ☒
하위 모듈	`sklearn.neighbors`	두 KNN 모델	`neighbors`에 **s**
하위 모듈	`sklearn.linear_model`	선형회귀 모델	`linear_model`의 밑줄
하위 모듈	`sklearn.model_selection`	훈련·테스트 분할	`model_selection`의 밑줄
하위 모듈	`sklearn.metrics`	MAE 등 평가 지표	`metrics` (복수)
클래스	`KNeighborsClassifier`	KNN 분류	`K` + `Neighbors` + `Classifier`
클래스	`KNeighborsRegressor`	KNN 회귀	`Regressor`, `Regression`이 아님
클래스	`LinearRegression`	선형·다항 회귀	`Linear` + `Regression`
함수	`train_test_split`	데이터 분할	단어마다 _
함수	`mean_absolute_error`	평균 절대 오차	`mean` + `absolute` + `error`
메서드	`.fit(X, y)`	학습	`fit` 소문자
메서드	`.score(X, y)`	점수	`score`, `scroe` ☒
메서드	`.predict(X)`	결과 예측	`predict`, `pre` ☒
메서드	`.predict_proba(X)`	분류 클래스별 확률	`proba`로 끝남
메서드	`.kneighbors(X)`	거리와 이웃 인덱스	`k` 소문자; `neighbors` 복수
설정	`n_neighbors`	이웃 수 k	**`n_`** + `neighbors`
속성	`.coef_`	학습한 계수	맨 끝 _ 필수
속성	`.intercept_`	학습한 절편	맨 끝 _ 필수
속성	`.shape`	배열의 모양	`shape()` ☒
배열 메서드	`.reshape(-1, 1)`	2차원 열벡터로 변경	`reshape`, 괄호 안은 `-1, 1`
배열 함수	`np.array()`	NumPy 배열 생성	`array`
배열 함수	`np.arange()`	범위 배열 생성	`arange` (`arrange` ☒)
배열 함수	`np.zeros()`	0으로 채우기	`zeros`
배열 함수	`np.ones()`	1로 채우기	`ones`
배열 함수	`np.concatenate()`	배열 연결	`concatenate` 철자 주의
배열 함수	`np.column_stack()`	**열 방향**으로 모으기	`column` + `_stack`

분류	정확한 철자	뜻·대표 사용법	암기 주의
배열 함수	`np.vstack()`	**행 방향**으로 쌓기	v + stack
배열 함수	`np.full()`	지정한 값으로 채우기	full
통계 함수	`np.mean()`	평균	mean
통계 함수	`np.std()`	표준편차	std
난수 함수	`np.random.seed()`	난수 시작 상태 고정	random.seed
난수 함수	`np.random.shuffle()`	제자리 섞기	shuffle
그래프 함수	`plt.scatter()`	산점도	scatter, sacater ☒
그래프 함수	`plt.plot()`	선·곡선	plot
그래프 함수	`plt.xlabel()`	x축 제목	xlabel
그래프 함수	`plt.ylabel()`	y축 제목	ylabel
그래프 함수	`plt.title()`	그래프 제목	title
그래프 함수	`plt.legend()`	범례	legend
그래프 함수	`plt.xlim()`	x축 구간	xlim
그래프 함수	`plt.show()`	그래프 출력	show
기본 함수	`zip()`	같은 순서 요소 묶기	zip
기본 함수	`len()`	길이	len
기본 함수	`print()`	출력	print
포맷	`.format()`	문자열 내 값 치환	{score:.2f}

반드시 구별할 4쌍

- **Classifier vs Regressor**: `KNeighborsClassifier`는 분류(0·1 같은 클래스), `KNeighborsRegressor`는 회귀(무게 같은 수치).
- **Regressor vs Regression**: KNN은 `KNeighborsRegressor()`지만 선형은 `LinearRegression()`.
- **predict vs predict_proba**: `predict()`는 최종 클래스/예측 숫자, `predict_proba()`는 **분류에서** 클래스별 확률.
- **coef_ vs intercept_**: `coef_`는 각 특성의 **기울기/계수**, `intercept_`는 **절편**. 둘 다 **학습 후 붙는 밑줄**을 빼먹지 않는다.

---

2. `import`: 라이브러리·패키지·클래스 정확히 쓰기

2-1. 다섯 노트북에 등장하는 import 7줄

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import KNeighborsRegressor
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
```

암기 방식: 아래 문장에서 빈칸을 가리고 7줄을 직접 쓴다.

목적	문장을 외울 때의 핵심
NumPy	`import numpy as np`
Matplotlib	`import matplotlib.pyplot as plt` (가운데 점 .)
KNN 분류	`from sklearn.neighbors import KNeighborsClassifier`
KNN 회귀	`from sklearn.neighbors import KNeighborsRegressor`
선형·다항회귀	`from sklearn.linear_model import LinearRegression`
자동 분할	`from sklearn.model_selection import train_test_split`
평균 절대 오차	`from sklearn.metrics import mean_absolute_error`

시험 함정

- `from sklearn.neighbor ...` ☒ → `neighbors` **s 포함**.
- `from sklearn.linear_model import LinearRegressor` ☒ → `LinearRegression`.
- `from sklearn.metrics import mean_absolute_errors` ☒ → `mean_absolute_error` (마지막 `error` 단수).
- `import matplotlib as plt` ☒ (노트북 사용 방식과 다름) → `import matplotlib.pyplot as plt`.
- `np`와 `plt`는 `as` 뒤 **임의로 붙인 별칭**이지만, 제공 노트북의 코드에서는 반드시 이 이름을 사용해야 한다.

2-2. `from ... import ...`와 `import ... as ...` 차이

```
from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier()      #

import numpy as np
a = np.array([1, 2, 3])      # np.
```

`KNeighborsClassifier()`의 괄호는 모델 객체를 만드는 호출이다. 일반적인 머신러닝 진행 순서는 **클래스 불러오기 → 객체 생성 → fit → score/predict**이다.

---

3. 파이썬 기본 구문: 리스트·인덱싱·반복·출력·문자열

3-1. 분류 데이터의 숫자와 정답 라벨

실제 1~3번 노트북은 도미 **35마리**, 빙어 **14마리**, 총 **49마리**를 사용한다.

```
length = bream_length + smelt_length
weight = bream_weight + smelt_weight
fish_target = [1] * 35 + [0] * 14
```

- `+` : 두 **리스트**를 이어 붙인다 (`np.array`끼리의 `+`는 원소별 덧셈이므로 구별).
- `[1] * 35` : 1을 **35개 반복**한 리스트.
- `[0] * 14` : 0을 **14개 반복**한 리스트.
- `1 =  , 0 =  ` (이 실습의 약속).
- `len(length)` → 49, `len(weight)` → 49, `len(fish_target)` → 49.
- 정답값 1 또는 0은 '도미일 확률' 자체가 아니라 **분류 클래스 라벨**이다.

3-2. `zip`과 리스트 컴프리헨션

```
fish_data = [[L, W] for L, W in zip(length, weight)]
```

읽는 순서:

1. `zip(length, weight)`는 같은 위치의 `(  ,  )` 짝을 만든다.
2. `for L, W in ...`는 한 쌍씩 꺼낸다.
3. `[L, W]`는 각 생선의 두 특성을 리스트로 만든다.
4. 바깥 `[...]`가 49개 생선 샘플을 모은다.
5. 결과 모양은 **49행 × 2열**이고, 첫 특성(0열)이 길이, 둘째 특성(1열)이 무게다.

[학습용 예시]

```
length = [10, 20]
weight = [100, 200]
print(list(zip(length, weight)))
# [(10, 100), (20, 200)]

print([[L, W] for L, W in zip(length, weight)])
# [[10, 100], [20, 200]]
```

3-3. 슬라이싱 `[:35]`와 `[35:]`

```
train_input = fish_data[:35]
test_input = fish_data[35:]
```

- `[:35]` = 인덱스 **0 ~ 34**, 35번 제외 = 35개.
- `[35:]` = 인덱스 **35 ~ 끝**, 14개.
- `[0:35]`와 `[:35]`는 같다.
- 파이썬 슬라이스 **끝 번호는 포함하지 않는다**.
- 이 데이터는 앞의 35개가 전부 도미이므로 이렇게 자르면 훈련은 도미만, 테스트는 빙어만 남는다 → 잘못된 평가의 예시(`score`가 `0.0`).

3-4. NumPy 배열 인덱싱: 가장 자주 나오는 문법

```
train_input[:, 0]       #    , 0 ( )
train_input[:, 1]       #    , 1 ( )
train_input[0, 0]       # 0  0
train_input[0, 1]       # 0  1
train_input[idx, 0]     # idx     0 (   )
train_input[idx, 1]     # idx     1 (   )
```

- `:` = 해당 축의 **전체**.
- `[ , ]`에서 **행이 먼저**.
- 열의 번호는 **0부터 시작**.
- `idx`가 2차원 인덱스 배열이면 `train_input[idx, 0]`도 2차원 결과가 될 수 있다. 산점도에서는 각 이웃 좌표가 맞도록 x·y 둘 다 동일한 `idx`를 사용한다.
- 파이썬 리스트에 바로 `fish_data[:, 0]`처럼 NumPy 형식으로 접근하면 오류가 난다. 먼저 `np.array(fish_data)` 등으로 바꿔야 한다.

3-5. `print`, `len`, 반복문

```
print(len(fish_data))
# 49

for n in [3, 5, 10, 42]:
    knr.n_neighbors = n
    knr.fit(train_input, train_target)
    print(knr.score(test_input, test_target))
```

- `for n in [...]` : 리스트 안 숫자를 순서대로 `n`에 넣는다.
- `:` 뒤에 반드시 **들여쓰기**.
- `print()`를 사용하면 값을 명시적으로 출력하며, 노트북에서 셀 마지막에 놓인 단독 표현식의 결과는 자동 표시될 수 있다.

3-6. 문자열 포맷: `.format()` 및 `:.2f`

```
train_score = 0.9804899950518966
label = 'train(score={score:.2f})'.format(score=train_score)
print(label)
# train(score=0.98)
```

- `{score}` : 대입할 항목 이름.
- `:.2f` : 소수점 아래 **두 자리** 고정 소수로 출력.
- `.format(score=train_score)` : 문자열의 `score` 자리에 변수 `train_score`를 넣는다.
- 그래프에서 `label='...'`은 `plt.legend()`와 조합해 범례에 표시한다.
- 원본 회귀 노트북에 있는 `'tarin'`은 **문자열 레이블의 오타**이고, 함수 이름이 아니다. 스스로 새 코드를 쓸 때는 `'train'`을 사용하면 된다.

---

4. 입력·정답 데이터 구성과 NumPy 함수

4-1. NumPy 기본 생성·구조 확인

표현	인수 설명	결과/특징
`np.array([1, 2, 3])`	리스트 → 배열	1차원, `(3,)`
`np.zeros(3)`	0을 3개	`[0., 0., 0.]`
`np.ones(3)`	1을 3개	`[1., 1., 1.]`
`np.full((2, 3), 9)`	**(행, 열), 채울 값**	2행 3열, 전부 9
`np.arange(5)`	0부터 5 직전	`[0,1,2,3,4]`
`np.arange(2, 7)`	2부터 7 직전	`[2,3,4,5,6]`
`np.arange(2, 10, 3)`	시작, 끝(미포함), 간격	`[2,5,8]`
`a.shape`	배열 모양 속성	예: `(3,)`, `(3, 2)`
`a.reshape(-1, 1)`	샘플 수 자동, 열 1개	1차원 → 2차원

실제 자료의 타깃 배열 생성 코드

```
fish_target = np.concatenate((np.ones(35), np.zeros(14)))
```

- `np.ones(35)` = 도미 라벨 1.0이 35개.
- `np.zeros(14)` = 빙어 라벨 0.0이 14개.
- `np.concatenate(( 1,  2))` = 첫 번째 배열 뒤에 두 번째 배열을 붙인다.
- `np.concatenate` 인수로 **여러 배열을 튜플/리스트에 묶어** 전달한 점을 주의.

4-2. `np.column_stack`: 열로 붙이기

```
fish_data = np.column_stack((length, weight))
```

- 1열(인덱스 0) = 길이, 2열(인덱스 1) = 무게.
- 길이 49개, 무게 49개 → `(49, 2)`.
- 다른 노트북에서는 `pred_target = np.column_stack((pred, test_target))`로 **예측값 / 실제값**을 나란히 보여준다. 열 순서 바꾸면 결과 열도 반대로 바뀜.

[학습용 예시]

```
print(np.column_stack(([1, 2, 3], [4, 5, 6])))
# [[1 4]
#  [2 5]
#  [3 6]]
```

4-3. `np.vstack`: 행(세로)으로 쌓기

온라인 보충 코드에 등장하는 정확한 철자다.

```
print(np.vstack(([1, 2, 3], [4, 5, 6])))
# [[1 2 3]
#  [4 5 6]]
```

`column_stack`은 **열 2개 × 행 3개**, `vstack`은 **행 2개 × 열 3개**가 된다는 차이를 실제 숫자로 구별한다.

4-4. `np.concatenate` vs `column_stack` vs `vstack`

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.concatenate((a, b))  # [1 2 3 4 5 6]   -> shape (6,)
np.column_stack((a, b)) # [[1 4],[2 5],[3 6]] -> shape (3,2)
np.vstack((a, b))       # [[1 2 3],[4 5 6]] -> shape (2,3)
```

기억: **단순 연결 = concatenate / 특성을 좌우 열로 = column_stack / 배열을 위아래 행으로 = vstack**.

4-5. 난수 인덱스: `arange` + `seed` + `shuffle`

```
input_arr = np.array(fish_data)
target_arr = np.array(fish_target)

np.random.seed(42)
index = np.arange(49)
np.random.shuffle(index)

train_input = input_arr[index[:35]]
train_target = target_arr[index[:35]]
test_input = input_arr[index[35:]]
test_target = target_arr[index[35:]]
```

1. `np.arange(49)`는 **0~48의 인덱스 49개**.
2. `np.random.seed(42)`는 실행할 때 동일한 난수 순서를 재현하도록 초기값을 고정.
3. `np.random.shuffle(index)`는 `index` 배열을 **제자리에서 섞음**. 반환값을 `index = np.random.shuffle(index)`라고 저장하면 `None`이 되므로 주의.
4. 입력(X)과 정답(y)에 **같은 `index[:35]` 또는 `index[35:]`**를 써야 짝이 안 틀어진다.
5. 위 수동 분할은 **훈련 35개/테스트 14개**이며, `train_test_split`을 기본값으로 실행한 결과와 개수가 다를 수 있다.

4-6. `mean`, `std`: 평균·표준편차

```
mean = np.mean(train_input, axis=0)
std = np.std(train_input, axis=0)
```

- `np.mean` = 평균(mean).
- `np.std` = 표준편차(standard deviation).
- `axis=0` = **각 열별** 통계값(이 실습에서 길이 평균, 무게 평균 각각).

- `axis=1` = **각 행별** 통계값(한 생선의 두 특성 사이 평균 등).
- 두 특성의 표준화에서는 반드시 **열별 평균과 표준편차**가 필요하므로 `axis=0`.
- NumPy 배열은 산술 연산에서 여러 행에 같은 열별 평균/표준편차를 자동 적용(**브로드캐스팅**)할 수 있다.

---

5. 훈련·테스트 분할과 데이터 배열의 `shape`

5-1. 암기할 공식 형태

```
from sklearn.model_selection import train_test_split

train_input, test_input, train_target, test_target = train_test_split(
    fish_data, fish_target, random_state=42
)
```

반환 순서가 **가장 중요**하다. 외울 때 한 줄로 암송:

train_input → test_input → train_target → test_target

이름	포함된 내용	쓰임
`train_input`	훈련 입력(X)	`fit` 입력, 훈련 점수
`test_input`	테스트 입력(X)	테스트 점수·새 예측
`train_target`	훈련 정답(y)	`fit` 정답, 훈련 점수
`test_target`	테스트 정답(y)	테스트 점수·MAE 비교

- `random_state=42`: 같은 무작위 분할 결과를 재현.
- 기본 `test_size`는 0.25 (25%)이고, **표본수에 따라 테스트 개수는 올림 처리될 수 있다**.
- 분류 데이터 49마리 → **훈련 36개, 테스트 13개** (실제 03 노트북).
- 회귀 데이터 56마리 → **훈련 42개, 테스트 14개** (실제 04·05 노트북).
- `random_state`를 정하지 않으면 매 실행마다 분할이 달라질 수 있다.

5-2. `stratify`: 클래스 비율 고려한 분할 — 온라인 보충

```
train_input, test_input, train_target, test_target = train_test_split(
    fish_data, fish_target,
    stratify=fish_target,
    random_state=42
)
```

- `stratify=fish_target`은 **전체 클래스의 비율**을 훈련·테스트에 비슷하게 유지하면서 나눈다.
- `fish_target`은 1(도미) 35개/0(빙어) 14개 → 도미 비율 35/49 ≈ 0.714.
- 제공된 보충 예시의 도미 비율: 훈련 약 `0.7222`, 테스트 약 `0.6923`.

- 수동 분할에서 도미만 훈련하고 빙어만 평가하던 문제를 막는 방법 중 하나.
- `stratify`는 **분류 클래스 비율**을 고려할 때 사용하는 선택 인수. 연속형 회귀의 실제 무게 배열을 그대로 `stratify=perch_weight`로 넣는 일반적인 방식은 아니다.

5-3. `shape`와 `reshape(-1, 1)`

```
print(perch_length.shape)    # (56,)
train_input, test_input, train_target, test_target = train_test_split(
    perch_length, perch_weight, random_state=42
)
print(train_input.shape, test_input.shape)  # (42,) (14,)
train_input = train_input.reshape(-1, 1)
test_input = test_input.reshape(-1, 1)
print(train_input.shape, test_input.shape)  # (42, 1) (14, 1)
```

- `(56,)` = 원소가 56개인 **1차원** (`,`도 주의).
- `(42, 1)` = **42행 × 1열**인 **2차원**.
- `-1`: 행 수를 데이터 개수에 맞게 **자동 계산**.
- `1`: 열을 1개로 만들라는 뜻.
- 단일 특성을 사용하는 scikit-learn 모델은 입력 `X`에 보통 **(샘플 수, 특성 수)의 2차원**을 요구한다.
- 회귀에서 정답 `y`는 **1차원** `(42,)` 형태로 유지. `X`만 2차원으로 만들어도 된다.
- `np.array([[50]])` = `(1, 1)` 2차원 한 샘플.
- `np.array([[25, 150]])` = `(1, 2)` 2차원 한 샘플.
- `np.array([50])`은 `(1,)`이므로 모델 예측 입력으로 그대로 사용하면 2차원 형식 오류가 날 수 있다.

5-4. `train_test_split` 전에 `reshape`해도 되나?

제공 노트북에서는 **1차원 농어 길이를 분할한 뒤** `train_input`, `test_input`에 `reshape(-1, 1)`을 쓴다. 두 과정의 의미는 다르다.

- **분할**: 모델 훈련/평가 데이터를 나누기.
- **reshape**: 각 데이터의 차원 모양을 모델 입력 형식에 맞추기.
- 둘을 혼동하면 빈칸 문제에서 `train_test_split` 대신 `reshape`를 쓰는 오류가 나온다.

---

6. 모든 모델의 공통 사용법: `fit`, `score`, `predict`

6-1. 세 줄은 무조건 쓸 수 있어야 한다

```
model.fit(train_input, train_target)        #
train_score = model.score(train_input, train_target)  #
```

```
test_score = model.score(test_input, test_target)      #
pred = model.predict(test_input)                       #
```

호출	정확한 인수 순서	뜻	반환
`model.fit(X, y)`	입력(X), 정답(y)	모델 학습	모델 객체 자체(일반적으로)
`model.score(X, y)`	입력(X), 실제 정답(y)	성능 평가	숫자 하나(float)
`model.predict(X)`	입력(X) 하나	X에 대한 예측	샘플 수에 맞는 배열

- `fit`과 `score`의 **첫 인수는 input, 둘째 인수는 target**.
- `predict`는 **정답 인수 없음** (예측할 X만 전달).
- `score`가 항상 '정확도'는 아니다. **분류는 정확도 / 회귀는 결정계수 R^2**.
- 1.0은 분류에서는 모두 맞춤(정확도 100%), 회귀에서는 주어진 평가 데이터에 대해 잔차가 0인 완벽한 예측(R^2=1)을 의미.
- 새로운 데이터에서의 성능을 보려면 훈련에 쓰지 않은 `test_input`, `test_target`으로 평가한다.

6-2. 각 모델의 형태를 한 눈에 비교

모델	생성	학습	점수 해석	예측 결과	분류 확률
KNN 분류	`KNeighborsClassifier()`	`fit(X, y)`	정확도	클래스 0·1	`.predict_proba(X)` 가능
KNN 회귀	`KNeighborsRegressor()`	`fit(X, y)`	R^2	연속적인 수치	해당 메서드 없음
선형·다항 회귀	`LinearRegression()`	`fit(X, y)`	R^2	연속적인 수치	해당 메서드 없음

분류 예시(실제 실습 코드 형태)

```
kn = KNeighborsClassifier()
kn.fit(fish_data, fish_target)
print(kn.score(fish_data, fish_target))
print(kn.predict([[30, 600]]))
```

회귀 예시(실제 실습 코드 형태)

```
knr = KNeighborsRegressor()
knr.fit(train_input, train_target)
print(knr.score(test_input, test_target))
print(knr.predict([[50]]))
```

두 예시의 변수명 `train_input` 등은 **해당 노트북에서 만들어진 데이터를 사용한다**.
서로 다른 노트북의 `train_input`을 혼동하지 말 것.

---

7. K-최근접 이웃 분류: 확률·이웃·이웃 수

7-1. 모델 생성과 분류 결과

```
from sklearn.neighbors import KNeighborsClassifier
kn = KNeighborsClassifier()  #      : 5
kn.fit(fish_data, fish_target)

one = [[30, 600]]
print(kn.predict(one))       # [1] →
print(kn.predict_proba(one)) # [[0. 1.]]
```

- `KNeighborsClassifier()`는 주변 가까운 **k개** 샘플의 클래스를 보고 분류.
- 기본 `n_neighbors` 값은 **5**.
- `one = [[30, 600]]`에서 **바깥 대괄호**는 샘플 1개, 안쪽은 길이·무게 특성 2개.
- `predict(one)` → `[1]`이라는 **분류 라벨**.
- `predict_proba(one)` → `[[0., 1.]]`이라는 **클래스별 확률**. 실제 수업 데이터 클래스 순서는 일반적으로 `0`, `1` 오름차순이며 첫 열이 빙어(0), 다음 열이 도미(1). 정확한 순서를 확인하는 속성은 `kn.classes_` (이름 자체는 노트북 주석에 등장).
- `predict_proba`의 **한 행 확률 합계는 1**(표준적인 분류 확률 출력).

7-2. `predict_proba` 해석을 단계적으로

실제 03 노트북, 표준화하기 전:

```
one = np.array([[25, 150]])
print(kn.predict(one))       # [0.]
print(kn.predict_proba(one)) # [[0.6 0.4]]
```

- 클래스 0(빙어) 확률 `0.6`, 클래스 1(도미) 확률 `0.4`.
- 가장 큰 확률은 0.6 → `predict(one)`은 `[0.]` (빙어).
- **표준화 후** 똑같은 원래 샘플을 동일한 평균·표준편차로 변환하여 `kn.predict(one_scaled)`를 수행하면 `[1.]`이 출력됨. 특성 크기를 맞춰 **이웃 거리가 바뀌었기 때문**.
- 예시 숫자는 `03_KNN_` 의 실제 출력이고, 다른 학습 데이터 또는 k에서는 달라질 수 있다.

7-3. 이웃 수 `n_neighbors`

```
kn = KNeighborsClassifier(n_neighbors=5)
#
```

```
kn49 = KNeighborsClassifier(n_neighbors=49)
kn49.fit(fish_data, fish_target)
```

- **생성자 인수**: `n_neighbors=` .
- **객체 속성 변경**: `kn.n_neighbors = 3` 후 필요하면 `.fit()`을 다시 실행.
- 도미 35, 빙어 14인 데이터 전체를 49개 이웃으로 분류하면 다수 클래스 1로 예측하는 쪽에 치우치며, **학습 정확도** `35/49` `0.7142857`.
- 01 노트북 마지막 코드는 `kn49.score(fish_data, fish_target)`에 이어 `print(35/49)`를 실행한다. 화면에 보이는 `0.714285714...`는 **`print(35/49)`의 출력**이고, `score()` 값을 `print`한 것은 아님. 이 점을 코드 결과 문제에서 구분.
- k가 클수록 결정 경계가 덜 세밀해지기 쉽고, 매우 작을수록 데이터의 국소 변화에 민감해질 수 있다.

7-4. `kneighbors`: 거리와 인덱스 두 개 반환

```
distances, idx = kn.kneighbors(one)
```

- `kneighbors` 철자: `k` + `neighbors`, 모든 글자 소문자.
- 반환 **첫 번째 = distances(거리)**, **두 번째 = idx(훈련 데이터 내부의 인덱스)**.
- 기본 이웃 수 5, 새 샘플 1개 → 거리 배열 모양 `(1, 5)`, 인덱스 배열 모양 `(1, 5)`.
- 거리는 '훈련 데이터에서 얼마나 가까운가'를 나타내고, 인덱스는 그 이웃이 `train_input`의 몇 번째 행인지 가리킨다.
- 원래 데이터 배열에서 뽑아내는 게 아니라 **해당 모델에 `fit()`한 훈련 입력 행 순서 기준**으로 인덱스를 해석해야 한다.

```
plt.scatter(train_input[:, 0], train_input[:, 1])
plt.scatter(one[0, 0], one[0, 1], marker='P')
plt.scatter(train_input[idx, 0], train_input[idx, 1], marker='^')
```

- `marker='P'`: 새 샘플 표기.
- `marker='^'`: 이웃 표기(삼각형).
- `one[0, 0]`, `one[0, 1]`는 NumPy 배열 기준. 파이썬 이중 리스트라면 `one[0][0]`, `one[0][1]` 사용.

---

8. 표준화(Z-score): `mean`, `std`, `axis`와 좌표

8-1. 반드시 암기할 표준화 식

표준점수(Z-score): `z = (  -  ) /`

```
mean = np.mean(train_input, axis=0)
std = np.std(train_input, axis=0)

train_scaled = (train_input - mean) / std
```

```
test_scaled = (test_input - mean) / std
one_scaled = (one - mean) / std
```

빈칸 핵심: `mean`, `std`, `axis=0`, 빼기 `-`, 나누기 `/` 그리고 평균·표준편차는 **훈련 데이터에서만 구한다**.

8-2. 왜 표준화를 하나?

이 실습은 생선의 길이(수십)와 무게(수백)의 숫자 규모가 크게 다르다. KNN이 거리로 이웃을 선택하면 숫자 규모가 큰 무게가 거리 계산에 더 큰 영향을 줄 수 있다. 평균 0, 표준편차 1 정도의 단위로 바꾸는 Z-score로 특성 간 크기를 맞춘다.

8-3. 훈련값만으로 평균·표준편차 계산

- `np.mean(train_input, axis=0)`과 `np.std(train_input, axis=0)` 사용.
- `np.mean(test_input, axis=0)`으로 테스트용 평균을 **별도로 만들지 않는다**. 그러면 테스트 데이터의 정보로 처리 기준을 만들게 돼 훈련/테스트 좌표 기준이 달라진다.
- `one_scaled`도 반드시 **같은 `mean`, `std`**로 변환.
- 입력 특성 X는 표준화하지만 **분류 라벨 0·1인 target은 표준화하지 않는다**(실제 03 노트북 주석의 강조점).
- 원본 샘플 `one`을 표준화 모델에 그대로 넣지 않고 `one_scaled`를 넣는다.

8-4. 실행 순서

```
kn.fit(train_scaled, train_target)
print(kn.score(train_scaled, train_target))
print(kn.score(test_scaled, test_target))
print(kn.predict(one_scaled))

distances, idx = kn.kneighbors(one_scaled)
plt.scatter(train_scaled[:, 0], train_scaled[:, 1])
plt.scatter(test_scaled[:, 0], test_scaled[:, 1])
plt.scatter(one_scaled[0, 0], one_scaled[0, 1], marker='P')
plt.scatter(train_scaled[idx, 0], train_scaled[idx, 1], marker='^')
plt.show()
```

사고 문제: "표준화 이후에도 기존의 `idx`와 `train_input`을 이용해 이웃을 표시하면?" → 모델이 **표준화된 데이터**로 학습한 상태라면 `kn.kneighbors(one_scaled)`로 새 거리/인덱스를 구하고, **표준화된 `train_scaled` 좌표**에 표시해야 현재 모델과 그래프가 일치한다.

---

9. K-최근접 이웃 회귀: 예측 · R^2 · MAE · 과대/과소적합

9-1. KNN 회귀 전체 흐름

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split

train_input, test_input, train_target, test_target = train_test_split(
    perch_length, perch_weight, random_state=42
)
train_input = train_input.reshape(-1, 1)
test_input = test_input.reshape(-1, 1)

knr = KNeighborsRegressor()
knr.fit(train_input, train_target)
print(knr.score(train_input, train_target))
print(knr.score(test_input, test_target))
```

- KNN **분류**는 가까운 이웃의 클래스(다수결 방식)를 예측.
- KNN **회귀**는 가까운 이웃의 타깃 수치(농어 무게)를 평균내서 예측(기본 균등 가중치).
- 사용한 특성 X는 농어 **길이 1개**, 타깃 y는 농어 **무게 1개**.
- `KNeighborsRegressor()` 기본 k는 5.

9-2. 회귀 `score` = 결정계수 R^2

```
R² = 1 - (  -  )²   / (  -    )²
```

- `score`가 `0.97`이면 '정확도 97%'라는 뜻이 **아니다**. 회귀의 결정계수 R^2다.
- 일반적으로 **1에 가까울수록** 평가 데이터의 변동을 잘 설명한다.
- 0은 항상 실제값 평균을 예측하는 기준 수준, 음수도 가능(기준보다 더 안 좋은 예측).
- KNN 회귀 노트북의 k=5 실제 점수: 훈련 `0.9698823289`, 테스트 `0.9928094061`.
- 이 실습에서는 훈련 점수가 테스트 점수보다 낮아서 **과소적합을 의심**하고 k를 3으로 줄여 확인한다. 단, 점수의 단순 대소 관계만으로 언제나 과소적합·과대적합을 확정할 수는 없고 일반화 성능까지 함께 봐야 한다.

9-3. 평균 절대 오차 MAE

```
from sklearn.metrics import mean_absolute_error

pred = knr.predict(test_input)
mae = mean_absolute_error(test_target, pred)
print(mae)
# 19.157142857142862  (  04  , k=5   )
```

- `mean_absolute_error` = **Mean Absolute Error**, 줄여서 MAE.

- 인수 순서: `mean_absolute_error(y_true, y_pred)` = **(실제 정답, 예측값)**.
- `test_target`이 실제 무게, `pred`가 예측 무게.
- `abs( - )`의 평균이므로 **0 이상**, 0이면 모든 예측이 정확.
- 이 실습에서 단위는 농어 무게와 같은 **g**로 생각할 수 있다.
- `score()`의 R^2는 단위 없는 상대적 성능 지표, MAE는 원래 타깃과 **동일 단위**의 절대오차 평균.

[학습용 예시]

```
print(mean_absolute_error([10, 20, 30], [12, 18, 27]))
# (2 + 2 + 3) / 3 = 2.3333333333333335
```

9-4. 예측값·실제값 나란히 보기

```
pred = knr.predict(test_input)
pred_target = np.column_stack((pred, test_target))
print(pred_target)
```

- 0열은 `pred`(예측값), 1열은 `test_target`(실제값).
- `np.column_stack` 철자와 이 순서를 동시에 암기.

9-5. k 변경과 과대·과소적합

```
knr.n_neighbors = 3
knr.fit(train_input, train_target)
print(knr.score(train_input, train_target))
print(knr.score(test_input, test_target))
```

제공된 결과(k=3):

- 훈련 R^2 = `0.9804899951`
- 테스트 R^2 = `0.9746459964`

해석:

- **k가 너무 크면** 지나치게 많은 이웃 평균을 내어 국소 패턴을 놓칠 수 있음(과소적합 경향).
- **k가 너무 작으면** 일부 개별 샘플에 지나치게 민감할 수 있음(과대적합 경향).
- `knr.n_neighbors = 3`을 **썼다고 바로 예전 예측값이 새 k로 바뀌는 것은 아님**. 제공 노트북처럼 `fit()`으로 다시 학습하고 점수를 확인.
- 코드 `for n in [3, 5, 10, 42]`는 각각 k를 바꿔 점수·산점도를 비교하는 반복문.

9-6. 새로운 샘플 예측과 KNN의 한계

```
sample = np.array([[50]])
knr.n_neighbors = 3
knr.fit(train_input, train_target)
pred = knr.predict(sample)
```

```
print(pred)  # [1033.33333333]

sample = np.array([[200]])
pred = knr.predict(sample)
print(pred)  # [1033.33333333]
```

- 제공된 `04_KNN_` 에서는 50cm와 200cm의 예측 결과가 **동일한 약 1033.33g**.
- 까닭: KNN 회귀는 기존 훈련 샘플의 타깃값을 평균해 예측하므로 훈련 길이 범위를 훨씬 넘어가도 **새로운 증가 추세를 계속 연장해 예측하지 못함**.
- 04 노트북의 마지막 '7주차' 표시 셀도 **KNN 외삽 한계 예시**로서 연결되는 내용이다. 별도 7주차 알고리즘 범위를 확장하지 않는다.

---

10. 단항 선형회귀: `LinearRegression`, `coef_`, `intercept_`

10-1. 암기할 문법

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(train_input, train_target)
print(lr.score(train_input, train_target))
print(lr.score(test_input, test_target))
print(lr.coef_, lr.intercept_)
```

- `LinearRegression`을 **`sklearn.linear_model`**에서 import.
- `LinearRegression()`은 선형 관계의 계수를 추정하는 객체.
- `.coef_` : 각 특성의 계수(입력 특성이 1개이면 원소 1개 배열).
- `.intercept_` : 절편(이 실습의 1차 모델에서는 스칼라).
- 둘 다 `.fit()` 후 얻는 학습 결과 속성, **괄호를 붙여 호출하는 함수가 아님**.

10-2. 1차 회귀식과 예측

```
y = a x + b
```

- `x` = 농어 길이, `y` = 예측 무게.
- `a = lr.coef_[0]` (기울기·계수).
- `b = lr.intercept_` (절편).

```
sample = np.array([[50]])
pred = lr.predict(sample)
print(pred, lr.coef_ * sample + lr.intercept_)
```

실제 05 노트북의 1차 회귀 결과:

결과	실제 출력에 근거한 값
훈련 R^2	0.939846333997604
테스트 R^2	0.8247503123313558
`lr.coef_`	[39.01714496]
`lr.intercept_`	-709.0186449535477
`lr.predict([[50]])`	[1241.83860323]

- 계수·절편은 무조건 위의 숫자로 고정된 것이 아니라 **이 노트북의 데이터와 `random_state=42`에 대응한 결과**다.
- 1차 직선은 음수 무게도 예측할 수 있다. 실제 05 노트북의 테스트 예측 배열에는 음수 값이 있으며, 이는 단순한 선형 모델의 한계를 보여준다.
- `lr.coef_[0] * 15 + lr.intercept_` → `x=15`에 대응한 직선의 y 좌표. 그래프 선분의 한 점.

10-3. 선형 직선 그래프

```
plt.scatter(train_input, train_target, label='train')
plt.scatter(test_input, test_target, label='test')
plt.scatter(sample, pred, marker='P', label='sample')
plt.plot(
    [15, 50],
    [lr.coef_[0] * 15 + lr.intercept_, lr.coef_[0] * 50 + lr.intercept_],
    label='linear_regression'
)
plt.legend()
plt.show()
```

- `scatter`는 관측값·예측값의 **점**.
- `plot`은 `x=15`와 `x=50` 두 점을 연결한 **직선**.
- 이 코드는 **1차 선형회귀로 fit한 직후** 사용해야 한다. 같은 `lr`을 뒤에서 2차 다항 특성으로 재학습한 상태에서는 `lr.coef_`의 항 개수가 달라진다.

---

11. 2차 다항회귀: 제곱 열·예측·곡선 그래프

11-1. '다항회귀'도 클래스는 `LinearRegression`

교수님 시험 범위는 **단항 1차 선형회귀 + 2차 다항회귀**까지. 여기서는 별도의 `PolynomialRegression` 클래스를 가져오지 않는다. **같은 `LinearRegression()`에** 입력 특성을 `[ ², ]`의 2열로 만들어 학습시킨다.

$y = a\,x^2 + b\,x + c$

- `a = lr.coef_[0]`: **제곱항** 계수.
- `b = lr.coef_[1]`: **1차항** 계수.

- `c = lr.intercept_`: **상수항/절편**.

11-2. 다항 특성 만들기: 가장 중요한 빈칸

```
train_poly = np.column_stack((train_input ** 2, train_input))
test_poly = np.column_stack((test_input ** 2, test_input))

lr.fit(train_poly, train_target)
print(lr.coef_, lr.intercept_)
```

- `** 2` = **제곱** (`^2` 아님; Python에서 `^`는 비트 XOR 연산).
- `np.column_stack((A, B))`에 **튜플 괄호 2중** 사용.
- 열 순서 **첫 번째 열 = x^2, 두 번째 열 = x**.
- 학습할 때 `train_poly`, 평가할 때 `test_poly`.
- 원래 농어 길이 데이터는 **1특성**, 다항 특성은 **2특성**이므로 입력 shape는 `(42, 2)` 및 `(14, 2)`가 됨.
- **타깃 `train_target`은 그대로** 사용. 정답 무게를 제곱하는 것이 아님.

11-3. 새로운 샘플도 동일한 형태로 변환

```
sample = np.array([[5]])   #         5cm
sample_poly = np.column_stack((sample ** 2, sample))
pred = lr.predict(sample_poly)
print(pred)                 # [33.61888862]
```

- `sample = [[5]]`를 바로 `lr.predict(sample)`로 넣으면 특성 개수가 안 맞음. 모델은 2개 특성 `[x², x]`를 기대.
- `sample_poly`는 `[[25, 5]]`가 된다.
- `5cm`라는 입력은 **오프라인 원본 실습**, `50cm` 예측은 **온라인 보충 셀**에 구분되어 있다.

```
sample_50 = np.array([[50]])
sample_50_poly = np.column_stack((sample_50 ** 2, sample_50))
pred_50 = lr.predict(sample_50_poly)
print(pred_50)             # [1573.98423528]
```

11-4. 2차 회귀곡선 그리기

```
point = np.arange(15, 51)   # 15~50:   51
curve = lr.coef_[0] * point ** 2 + lr.coef_[1] * point + lr.intercept_

plt.scatter(train_input, train_target, label='train')
plt.scatter(test_input, test_target, label='test')
plt.plot(point, curve, label='polynomial regression')
plt.scatter(sample_50[0, 0], pred_50[0], marker='^', s=120, label='50cm prediction')
plt.xlabel('length')
plt.ylabel('weight')
```

```
plt.legend()
plt.show()
```

- `np.arange(15, 51)`은 **15부터 50까지 총 36개**.
- `curve`는 해당 36개 x에 대해 이차식으로 구한 y 값의 배열.
- `plt.plot(point, curve)`가 직선이 아닌 **곡선**처럼 보인다.
- `sample_50[0, 0] = 50`, `pred_50[0]`는 50cm 예측값.

11-5. 실제 다항회귀 결과(05 노트북)

항목	출력
훈련 R^2	0.9706807451768623
테스트 R^2	0.9775935108325122
`lr.coef_`	[1.01433211, -21.55792498]
`lr.intercept_`	116.05021078278264
5cm 농어 예측	[33.61888862]
50cm 농어 예측	[1573.98423528]

- 1차 회귀에서 같은 변수 `lr`을 **다항 특성으로 다시** `.fit()`하면 이후 `.coef_`, `.intercept_`, `.predict()`는 **2차 다항회귀 기준**이 된다.
- 선형회귀와 다항회귀 모두 `LinearRegression`을 사용하더라도 **학습에 넣은 특성 열이 다르므로 결과 계수 수와 그래프 모양이 달라진다**.
- 직선 예측 50cm 약 **1241.84g**, 2차 곡선 예측 50cm 약 **1573.98g**, KNN 회귀(k=3) 예측 50cm 약 **1033.33g**. 서로 다른 모델과 훈련 방식에서 나온 결과임을 구분.

---

12. Matplotlib 그래프 함수·옵션 한 글자씩 암기

12-1. 기본 8종

목적	정확한 코드	해석
그래프 불러오기	`import matplotlib.pyplot as plt`	`pyplot`을 `plt`로 사용
점 찍기	`plt.scatter(x, y)`	산점도
선/곡선	`plt.plot(x, y)`	데이터를 순서대로 연결
x축 이름	`plt.xlabel('length')`	길이 축
y축 이름	`plt.ylabel('weight')`	무게 축
제목	`plt.title('KNN regressor')`	그래프 위 제목
범례	`plt.legend()`	각 플롯의 `label` 보여주기

목적	정확한 코드	해석
x축 범위	`plt.xlim(0, 1000)`	표시 x 범위를 0~1000으로 제한
출력	`plt.show()`	그래프 표시

12-2. 플롯 선택 옵션

```
plt.scatter(x, y, marker='^', s=120, label='new fish')
```

- `marker` = 마커의 모양(실습: `'P'`, `'^'`).
- `s` = 산점도 마커 크기(포인트² 단위).
- `label` = `plt.legend()`에서 보일 이름.
- `plt.xlabel` / `plt.ylabel`은 **각각 축 이름**. `plt.title`은 **전체 그래프 제목**.
- `plt.xlim(0, 1000)`은 **x축** 범위를 바꾸는 것, `plt.ylim`과 혼동하지 말 것. 원본 03 노트북은 x축 표시 범위를 넓혀 거리 감각이 왜곡될 수 있음을 보여주는 흐름이다.

12-3. 어떤 데이터를 가로·세로에 넣는가?

```
#  · :  = ,  =
plt.scatter(bream_length, bream_weight)
plt.scatter(smelt_length, smelt_weight)

# NumPy 2    : 0 = , 1 =
plt.scatter(train_input[:, 0], train_input[:, 1])

#  :  = ,  =
plt.scatter(train_input, train_target)

#    :  = ,  =
plt.scatter(test_input, pred)

#       :  =   ,  =    (    )
plt.scatter(pred, test_target)

# 1 /2   : x   , y
plt.plot(point, curve)
```

주의: `plt.scatter`의 x, y 인수는 **같은 수의 좌표값**이어야 한다. 2차원 생선 특성 배열 전체를 그대로 x 인수에 넣을 경우 의도한 도미/빙어 길이-무게 산점도가 되지 않는다. `[:, 0]`와 `[:, 1]`로 분리한다.

---

13. 자료별로 구분해야 할 실제 출력·값

이 부분은 **예측값과 정답이 달라지는 순간** 및 **어떤 셀을 먼저 실행해야 하는지**를 염두에 두고 외운다. 아래는 첨부된 노트북에 저장된 실행 결과를 기준으로 한다.

자료·상황	코드/변수	결과
01 분류 데이터 수	`len(length)`, `len(weight)`	`49`, `49`
01 전체 데이터로 KNN 분류 학습·평가	`kn.score(fish_data, fish_target)`	`1.0`
01 새 생선 길이30, 무게600	`kn.predict([[30,600]])`	`[1]`
01 새 생선 클래스별 확률	`kn.predict_proba([[30,600]])`	`[[0. 1.]]`
01 k=49 비율 출력	`print(35/49)`	`0.7142857142857143`
02 원래 순서 그대로 35/14 분할	`kn.score(test_input, test_target)`	`0.0`
02 난수 인덱스 섞은 뒤 재학습	훈련·테스트 점수	`1.0` / `1.0`
02 수동 난수 분할 후 입력 모양	`train_input.shape`, `test_input.shape`	`(35, 2)`, `(14, 2)`
03 NumPy 분류 데이터	`fish_data.shape` (구성에서 도출)	`(49, 2)`
03 자동 분할	`train_input.shape`, `test_input.shape`	`(36, 2)`, `(13, 2)`
03 분할 정답 모양	`train_target.shape`, `test_target.shape`	`(36,)`, `(13,)`
03 비표준화 샘플 [25,150]	`kn.predict(one)`	`[0.]`
03 비표준화 샘플 확률	`kn.predict_proba(one)`	`[[0.6 0.4]]`
03 표준화 후 같은 샘플	`kn.predict(one_scaled)`	`[1.]`
04 농어 데이터 개수	`perch_length.shape`	`(56,)`
04 농어 분할 후 reshape	`train_input.shape`, `test_input.shape`	`(42, 1)`, `(14, 1)`
04 k=5 KNN 회귀	훈련/테스트 R^2	`0.9698823` / `0.9928094`
04 k=5 KNN 회귀	MAE	`19.157142857...`
04 k=3 KNN 회귀	훈련/테스트 R^2	`0.9804900` / `0.9746460`
04 k=3 농어 50cm	예측	`[1033.33333333]`
04 k=3 농어 200cm	예측	`[1033.33333333]`
05 단항 선형회귀	훈련/테스트 R^2	`0.9398463` / `0.8247503`
05 단항 선형회귀	`coef_`, `intercept_`	`[39.01714496]`, `-709.01864495...`
05 단항 선형회귀 50cm	예측	`[1241.83860323]`
05 2차 다항회귀	훈련/테스트 R^2	`0.9706807` / `0.9775935`

자료·상황	코드/변수	결과
05 2차 다항회귀	`coef_`, `intercept_`	`[1.01433211, -21.55792498], 116.05021078...`
05 2차 다항회귀 5cm	예측	`[33.61888862]`
05 2차 다항회귀 50cm	예측	`[1573.98423528]`

숫자 암기 우선순위: `49=35+14`, `56=42+14`, `knn [0]→ [1]`, **모델별 `score` 의미**, k 변경 결과, **1차 vs 2차 `coef_` 구조**를 먼저 암기한다. 소수점 긴 출력은 수업 자료의 특정 실행 환경·분할에 따른 값이므로, 시험에서 다른 코드/데이터가 주어지면 **제시된 코드의 실행 논리**로 계산해야 한다.

---

14. 틀리기 쉬운 철자·인수·문제 함정 총정리

틀린 표현·오해	정확한 표현·판단	이유
`KNeighborClassifier`	`KNeighborsClassifier`	`Neighbors` 복수
`KNeighborsRegression()`	`KNeighborsRegressor()`	모델 클래스 명
`LinearRegressor()`	`LinearRegression()`	클래스 명
`train_test_splite`	`train_test_split`	뒤는 `split`
`mean_absolute_errors`	`mean_absolute_error`	끝은 `error`
`np.arrange()`	`np.arange()`	r 하나: `arange`
`np.concatanate()`	`np.concatenate()`	정확히 `concatenate`
`np.columnstack()`	`np.column_stack()`	밑줄
`np.v_stack()`	`np.vstack()`	붙여 쓰기
`np.full(9, (2,3))`	`np.full((2,3), 9)`	shape가 먼저
`np.random.shuffer()`	`np.random.shuffle()`	`shuffle`
`np.random.sheed()`	`np.random.seed()`	`seed`
`plt.sacater()`	`plt.scatter()`	`scatter`
`plt.scattar()`	`plt.scatter()`	`scatter`
`plt.label('x')`	`plt.xlabel('x')`	x/y 구분
`model.pre(X)`	`model.predict(X)`	정확한 메서드
`model.predict_proba(X)` for 회귀	**분류**에서만 이용	회귀는 무게 수치 예측
`kn.kneighbor(one)`	`kn.kneighbors(one)`	복수
`kn.neighbors=3`	`kn.n_neighbors = 3`	접두어 `n_`
`kn.fit(target, input)`	`kn.fit(input, target)`	순서 X→y
`kn.score(test_target, test_input)`	`kn.score(test_input, test_target)`	순서 X→y

틀린 표현·오해	정확한 표현·판단	이유
`mean_absolute_error(pred, target)`	권장 `mean_absolute_error(target, pred)`	API 순서: 실제값→예측값; MAE는 대칭이라 숫자는 같아도 인수 역할을 외우기
`lr.coef`	`lr.coef_`	학습된 속성에 _
`lr.intercept`	`lr.intercept_`	학습된 속성에 _
`lr.coef_()`	`lr.coef_`	속성, 함수 아님
`arr.shape()`	`arr.shape`	속성, 함수 아님
`X.reshape(1, -1)`	`X.reshape(-1, 1)`	여러 샘플의 단일 특성 열 만들기
`X ** 2` 대신 `X ^ 2`	`X ** 2`	`**` 제곱, `^` 비트 XOR
`np.arange(15,50)`으로 50 포함 기대	`np.arange(15, 51)`	stop 값 미포함
`train_poly = np.column_stack((train_input, train_input**2))`	**수업 자료는** `(train_input**2, train_input))`	열 순서가 `coef_[0]`, `coef_[1]` 의미 결정
2차 회귀 모델에 `[[50]]` 바로 예측	`[[2500, 50]]` 특성으로 바꿔 예측	모델은 열 2개를 기대
테스트 데이터만 다른 평균·표준편차로 나눔	훈련 데이터 `mean`, `std` 재사용	같은 스케일 필요
타깃 라벨까지 표준화	입력 특성 X만 표준화	도미/빙어 라벨 의미 유지
`score()`가 모든 모델의 정확도	회귀는 **R^2**	분류와 평가 지표 구분
`predict_proba`의 첫 열이 무조건 도미	이 실습은 **0(빙어) → 1(도미)**	클래스 순서에 따라 바뀜
`index = np.random.shuffle(index)`	`np.random.shuffle(index)`	`shuffle`은 제자리 수정
그래프에 `idx`의 열을 섞음	X,Y 둘 다 동일 `idx` 사용	같은 이웃 좌표 필요
`plt.scatter`로 회귀 곡선만 표현	`plt.plot(point, curve)`	선/곡선은 `plot`
무작위 섞은 뒤 과거 `fit` 모델 그대로 사용	새 `train_input`, `train_target`으로 다시 `fit`	재학습해야 새 데이터 반영
파일마다 같은 `train_input`이라고 동일 데이터	**01~03 = 생선 길이·무게, 04~05 = 농어 길이**	노트북 별 셀 실행 흐름 다름

코드의 모양을 보면 메서드까지 역으로 맞추는 연습

- **정답과 함께 데이터를 전달**하고 '훈련': `.fit(X, y)`.
- **정답과 함께 데이터를 전달**하고 '점수': `.score(X, y)`.
- **입력만 전달**하고 '0/1 또는 무게 예측': `.predict(X)`.
- **입력만 전달**하고 '각 클래스 확률': `.predict_proba(X)` (분류).

- **입력만 전달**하고 '(거리, 인덱스)' 반환: `.kneighbors(X)`.
- **정답 배열, 예측 배열**로 '평균 무게 오차': `mean_absolute_error(y_true, y_pred)`.
- **계수·절편 얻기**: `lr.coef_`, `lr.intercept_`.

---

15. 시험장에서 응용하는 완성 코드 패턴

긴 코드를 통째로 쓰라는 문제가 아니라도, **빠진 1~2줄이 어느 단계에 속하는지**를 알아야 빈칸 문제를 풀 수 있다. 다음은 **암기용 재구성 예시**이며 노트북의 모든 데이터 정의 부분을 반복하지는 않았다.

패턴 A. KNN 분류 + 테스트 + 확률

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

train_input, test_input, train_target, test_target = train_test_split(
    fish_data, fish_target, stratify=fish_target, random_state=42
)
kn = KNeighborsClassifier(n_neighbors=5)
kn.fit(train_input, train_target)
print(kn.score(train_input, train_target))
print(kn.score(test_input, test_target))
print(kn.predict([[30, 600]]))
print(kn.predict_proba([[30, 600]]))
```

- `fish_data`는 2개 특성인 2차원 배열/리스트, `fish_target`은 정답 배열.
- 샘플이 `[ , ]` 특성 2개라서 `[[30,600]]`.
- 새 샘플의 예측값은 **현재 분할·모델 학습 상태**에 따라 달라질 수 있으므로 이 재구성 예시에는 숫자 출력을 고정하지 않음.

패턴 B. Z-score KNN 분류 + 이웃

```
mean = np.mean(train_input, axis=0)
std = np.std(train_input, axis=0)
train_scaled = (train_input - mean) / std
test_scaled = (test_input - mean) / std
one = np.array([[25, 150]])
one_scaled = (one - mean) / std

kn.fit(train_scaled, train_target)
print(kn.predict(one_scaled))
distances, idx = kn.kneighbors(one_scaled)
plt.scatter(train_scaled[:, 0], train_scaled[:, 1])
```

```
plt.scatter(one_scaled[0, 0], one_scaled[0, 1], marker='P')
plt.scatter(train_scaled[idx, 0], train_scaled[idx, 1], marker='^')
plt.show()
```

패턴 C. KNN 회귀 + MAE + k 변경

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error

knr = KNeighborsRegressor(n_neighbors=5)
knr.fit(train_input, train_target)
pred = knr.predict(test_input)
print(knr.score(test_input, test_target))
print(mean_absolute_error(test_target, pred))

knr.n_neighbors = 3
knr.fit(train_input, train_target)
print(knr.predict([[50]]))
```

- 여기 `train_input`, `test_input`은 **농어 길이 단일 특성**으로 사전에 `reshape(-1,1)`을 마친 상태가 전제.

패턴 D. 1차 선형회귀 → 2차 다항회귀

```
from sklearn.linear_model import LinearRegression

lr = LinearRegression()
lr.fit(train_input, train_target)
print(lr.coef_, lr.intercept_)
print(lr.predict([[50]]))

train_poly = np.column_stack((train_input ** 2, train_input))
test_poly = np.column_stack((test_input ** 2, test_input))
lr.fit(train_poly, train_target)
print(lr.coef_, lr.intercept_)

sample = np.array([[50]])
sample_poly = np.column_stack((sample ** 2, sample))
print(lr.predict(sample_poly))
```

- 1차 학습은 `train_input`(특성 1개), 2차 학습은 `train_poly`(특성 2개).
- `lr.fit()`을 두 번 실행하므로 **두 번째 fit 이후의 lr은 다항 특성으로 학습된 모델**이다.

---

16. 실전 빈칸·결과 예측·오류 찾기 100문제 **풀이 규칙**: 답을 바로 보지 말고 17.

을 접어 두고 푼다. 교수가 공지한 형식에 맞춰 **정확한 철자·구문·실행 결과**를 쓰는 연습이다. 여기에는 **노트북 실제 내용에서 변형한 문제와** [ ] **문제**가 함께 있다. 실제 출제 문제라는 뜻은 아니다.

___
###
A.
라이브러리·모델·메서드
철자
(1~20)

1. NumPy를 `np`로 사용하도록 불러오는 정확한 한 줄을 쓰시오.
2. Matplotlib의 `pyplot`을 `plt`로 사용하도록 불러오는 한 줄을 쓰시오.
3. `KNeighborsClassifier`를 불러오는 import 한 줄을 쓰시오.
4. `KNeighborsRegressor`를 불러오는 import 한 줄을 쓰시오.
5. `LinearRegression`을 불러오는 import 한 줄을 쓰시오.
6. `train_test_split`을 불러오는 import 한 줄을 쓰시오.
7. `mean_absolute_error`를 불러오는 im-

###
B.
파이썬·NumPy
데이터
구성과
배열
(21~40)

---

21. `bream_length`와 `smelt_length` 두 **파이썬 리스트**를 합쳐 `length`에 저장하는 코드는?

22. 도미 35개에 1, 빙어 14개에 0을 할당하는 파이썬 리스트 식은?

23. 35개 + 14개의 전체 샘플 수는?

24. `length`와 `weight`를 짝지어 `fish_data`를 만드는 리스트 컴프리헨션 전체 식을 쓰시오.

25. `np.array`를 이용해 `fish_data`를 NumPy 배열인 `input_arr`로 만드는 코드는?

26. `np.ones(35)`의 원소 개수와 값은?

27. `np.zeros(14)`의

###

C.

데이터

분할·shape·표준화

(41~60)

41. `train_test_split`의 결과를 받을 변수 4개를 **정확한 순서**로 쓰시오.

42. `fish_data`, `fish_target`을 `random_state=42`로 자동 분할하는 대입문을 쓰시오.

43. 분류의 클래스 비율을 고려해 분할하는 옵션의 정확한 이름과 사용법은?

44. 분류 자료 49개를 기본 설정 `train_test_split`으로 나누면 훈련/테스트 샘플 개수는?

45. 회귀 자료 56개를 기본 설정으로 나누면 훈련/테스트 샘플 개수는?

46. 수동 슬라이스 `fish_data[:35]`는 몇 번 인덱스까지

D. 분류·이웃·회귀·평가 (61~80)

61. KNeighborsClassifier()의 기본 이웃 수는?

62. KNeighborsRegressor()의 기본 이웃 수는?

63. kn.predict([[30, 600]])의 실제 01 노트북 결과는?

64. 그 결과 [1]에서 클래스 1은 도미/빙어 중 무엇인가?

65. kn.predict_proba([[30,600]])의 실제 01 노트북 결과는?

66. 분류 확률 [[0.6, 0.4]]의 첫 확률이 클래스 0에 대응한다면 최종 예측 클래스는?

67. distances, idx = kn.kneighbors(one)에서 첫 반환값과 두 번째 반환값의

E. 선형·다항회귀·그래프·실행 결과 (81~100)

81. 단항 선형회귀의 수학식 y = ?를 a, x, b로 쓰시오.

82. lr.coef_와 lr.intercept_는 각각 무엇을 의미하나?

83. 1특성 선형회귀에서 기울기 숫자 하나에 접근하는 식은?

84. 05 노트북의 1차 선형회귀 50cm 농어 예측값은 약 얼마인가? (1033.33, 1241.84, 1573.98 중 택1)

85. 2차 다항회귀에서도 사용하는 scikit-learn 클래스 이름은?

86. train_poly를 길이 제곱과 길이 두 특성으로 만드는

17. 100문제 정답과 해설

정답만 보고 끝내지 말고, **한 글자라도 철자가 달랐던 번호**는 14. 을 다시 확인한다.

A 정답: 1~20

번호	정답	핵심 설명
1	`import numpy as np`	`numpy`, `np`
2	`import matplotlib.pyplot as plt`	점 `.`으로 이어진 `matplotlib.pyplot`
3	`from sklearn.neighbors import KNeighborsClassifier`	분류 클래스, `neighbors` 복수
4	`from sklearn.neighbors import KNeighborsRegressor`	회귀 클래스, `Regressor`
5	`from sklearn.linear_model import LinearRegression`	`linear_model`
6	`from sklearn.model_selection import train_test_split`	`model_selection`
7	`from sklearn.metrics import mean_absolute_error`	`metrics`, `absolute`
8	`kn = KNeighborsClassifier(n_neighbors=5)`	`n_neighbors`
9	`knr = KNeighborsRegressor(n_neighbors=3)`	`Regressor`
10	`lr = LinearRegression()`	괄호 필요
11	`fit`	`kn.fit(train_input, train_target)`
12	`score`	분류 정확도
13	`predict`	클래스 예측
14	`predict_proba`	클래스별 확률
15	`kneighbors`	거리·인덱스
16	`n_neighbors`	`kn.n_neighbors = 3`
17	`coef_`	끝 밑줄 포함
18	`intercept_`	끝 밑줄 포함

번호	정답	핵심 설명
19	`shape`	`shape()` 아님
20	KNN **분류**	회귀용 메서드가 아님

B 정답: 21~40

번호	정답	핵심 설명
21	`length = bream_length + smelt_length`	리스트 이어 붙이기
22	`[1] * 35 + [0] * 14`	도미1/빙어0
23	`49`	35+14
24	`fish_data = [[L, W] for L, W in zip(length, weight)]`	각 샘플 `[ , ]`
25	`input_arr = np.array(fish_data)`	리스트→배열
26	35개, 원소가 모두 1(실제 float `1.0`)	`ones`
27	14개, 원소가 모두 0(실제 float `0.0`)	`zeros`
28	`fish_target = np.concatenate((np.ones(35), np.zeros(14)))`	중첩 괄호
29	`np.column_stack((length, weight))`	열 방향
30	`np.vstack(([1,2,3], [4,5,6]))`	행 방향
31	`np.full((2, 3), 9)`	shape → 값
32	`[0, 1, 2, 3, 4]`	끝 5 제외
33	시작 `15`, 마지막 `50`, 개수 `36`	`51` 제외
34	`[2, 5, 8]`	step=3
35	`0`, `48`	49 미포함
36	`seed`	`np.random.seed(42)`
37	`shuffle`	`np.random.shuffle(index)`
38	안 됨. 반환값은 `None`, 배열 자체를 바꿈	제자리 셔플
39	`train_input[:, 0]`	모든 행의 0열
40	`(3, 2)`	행 3개, 열 2개

C 정답: 41~60

번호	정답	핵심 설명
41	`train_input, test_input, train_target, test_target`	입력 2개 → 정답 2개
42	`train_input, test_input, train_target, test_target = train_test_split(fish_data, fish_target, random_state=42)`	분할 함수 4개 반환
43	`stratify=fish_target`	클래스 비율 유지(온라인 보충)
44	훈련 36, 테스트 13	실제 03 결과
45	훈련 42, 테스트 14	실제 04·05 결과
46	34번 인덱스	끝 35 제외
47	35번 인덱스	35부터 끝
48	`0.0`	도미만 훈련, 빙어만 테스트
49	1차원	`(56,)`
50	`train_input = train_input.reshape(-1, 1)`	자동 행, 열 1개
51	`np.array([[50]])`	shape `(1,1)`
52	`np.array([[25, 150]])`	shape `(1,2)`
53	각 **열**별	길이·무게별 평균
54	각 특성 **열**별 표준편차	`std`
55	`train_input`	학습 데이터 기준
56	`train_input, mean, std`	`(train_input - mean) / std`
57	`test_input, mean, std`	훈련 통계값 재사용
58	`one, mean, std`	새 샘플에도 동일 변환
59	`one_scaled`	표준화 모델이기 때문
60	아니오	분류 정답 0·1은 그대로

D 정답: 61~80

번호	정답	핵심 설명
61	`5`	KNN 분류 기본 k
62	`5`	KNN 회귀 기본 k
63	`[1]`	01 실제 결과

번호	정답	핵심 설명
64	도미	이 실습에서 1=도미
65	`[[0. 1.]]`	0(빙어), 1(도미) 확률
66	`0`	더 큰 0.6은 클래스 0 쪽
67	`distances`: 거리, `idx`: 훈련 샘플 인덱스	반환 순서 필수
68	`(1, 5)`	샘플 1개 × 이웃 5개
69	`35/49` (약 `0.7142857`)	도미 다수 클래스
70	정확도(맞은 비율)	분류 점수
71	결정계수 R^2	회귀 점수
72	가능	평균 예측보다 오차가 클 때
73	이웃 타깃의 **평균**	기본 균등 가중치 KNN 회귀
74	`mean_absolute_error(y_test, y_pred)`	실제값 → 예측값
75	`7/3  2.3333`	(2+2+3)/3
76	0열: 예측 `pred`, 1열: 실제 `test_target`	`column_stack` 인수 순서
77	`0.99` (실제 약 `0.9928094`)	04 k=5 테스트 R^2
78	`knr.n_neighbors = 3`	바꾼 뒤 `.fit` 재실행
79	과소적합 경향	지나치게 평탄해짐
80	가까운 훈련 샘플의 타깃 평균으로만 예측하여 학습 범위 밖 증가 추세를 외삽하지 못함	KNN 회귀의 한계

E 정답: 81~100

번호	정답	핵심 설명
81	`y = a*x + b`	직선
82	`coef_`: 계수·기울기 / `intercept_`: 절편	학습된 속성
83	`lr.coef_[0]`	1특성 첫 계수
84	`1241.84`	05 단항 50cm
85	`LinearRegression`	다항 특성을 직접 생성
86	`train_poly = np.column_stack((train_input ** 2, train_input))`	첫 열 제곱
87	`test_poly = np.column_stack((test_input ** 2, test_input))`	같은 순서

번호	정답	핵심 설명
88	x^2의 계수	a
89	x의 계수	b
90	상수항·절편 c	intercept_
91	sample_poly = np.column_stack((sample ** 2, sample))	입력 샘플도 특성 변환
92	[[25, 5]]	5^2=25
93	lr.predict(sample_poly)	특성 2열 전달
94	1573.98	05 다항 50cm
95	**	거듭제곱
96	15, 16, ..., 50	51 제외
97	**, intercept_	이차식 연산자와 절편
98	plt.scatter	‘scatter’ 정확한 철자
99	plt.plot	선·곡선
100	plt.legend() 다음 plt.show()	범례 뒤 출력

---

18. 최종 암기 체크리스트

18-1. 7개 import를 빈 종이에 정확히 쓸 수 있는가?

- ☐ `import numpy as np`
- ☐ `import matplotlib.pyplot as plt`
- ☐ `from sklearn.neighbors import KNeighborsClassifier`
- ☐ `from sklearn.neighbors import KNeighborsRegressor`
- ☐ `from sklearn.linear_model import LinearRegression`
- ☐ `from sklearn.model_selection import train_test_split`
- ☐ `from sklearn.metrics import mean_absolute_error`

18-2. 데이터 구조·전처리를 코드로 쓸 수 있는가?

- ☐ `zip`, 리스트 컴프리헨션 `[[L,W] for ...]`
- ☐ `[1]*35 + [0]*14`, `np.ones`, `np.zeros`, `np.concatenate`
- ☐ `np.array`, `np.column_stack`, `np.vstack`, `np.full`, `np.arange`
- ☐ `np.random.seed`, `np.random.shuffle`, 같은 인덱스로 X/y 섞기
- ☐ `[:35]`와 `[35:]`의 정확한 범위
- ☐ `train_test_split` 변수 반환 순서 + `stratify` + `random_state`
- ☐ `.shape` 및 `.reshape(-1, 1)`과 `[[50]]`의 차원
- ☐ `[:,0]`, `[:,1]`, `[0,0]`, `[idx,0]` 구별
- ☐ `np.mean(..., axis=0)` 및 `np.std(..., axis=0)`
- ☐ `(train_input-mean)/std`, `(test_input-mean)/std`, `(one-mean)/std`

18-3. 모델을 정확히 고를 수 있는가?

- ☐ KNN 분류: `KNeighborsClassifier()` → 클래스 0/1
- ☐ KNN 회귀: `KNeighborsRegressor()` → 무게 평균
- ☐ 선형·다항회귀: `LinearRegression()` → 계수·절편
- ☐ 공통 `fit(X,y)`, `score(X,y)`, `predict(X)`의 괄호와 인수
- ☐ 분류 `predict_proba(X)`, KNN `kneighbors(X)`, `n_neighbors`
- ☐ `score`: 분류 정확도 vs 회귀 R^2
- ☐ MAE: `mean_absolute_error(test_target, pred)`
- ☐ k가 너무 크거나 작을 때의 과소/과대적합 **경향**
- ☐ KNN 회귀가 50cm, 200cm에 비슷한 결과를 낸 이유

18-4. 선형·다항회귀 구문을 눈 감고 쓸 수 있는가?

- ☐ `lr.coef_` vs `lr.intercept_` (밑줄 포함)
- ☐ 1차 `y=ax+b`와 `lr.coef_[0] * x + lr.intercept_`
- ☐ `train_poly = np.column_stack((train_input**2, train_input))`
- ☐ `test_poly = np.column_stack((test_input**2, test_input))`
- ☐ `sample_poly = np.column_stack((sample**2, sample))`
- ☐ `lr.coef_[0]`은 x^2, `lr.coef_[1]`은 x, `lr.intercept_`는 상수항
- ☐ `point = np.arange(15, 51)`과 이차식 `curve` 계산
- ☐ `plt.scatter`(점) vs `plt.plot`(선·곡선)

18-5. 시험 직전 실제 연습 방법

1. **1회독**: 이 문서의 1~15장을 읽으며 7개 import와 함수 철자를 손으로 적는다.
2. **2회독**: 16장의 **1~100번 문제를 답안 없이** 빈 종이 또는 `.md`에 직접 쓴다. 코드가 맞더라도 대·소문자·밑줄·괄호를 검사한다.
3. **오답 정리**: 틀린 번호의 답을 보고 17장의 정답을 3번씩 다시 쓰되, 특히 `fit/score/predict`, `coef_/intercept_`, `column_stack/vstack`을 구분한다.
4. **3회독**: 15장의 네 가지 코드 패턴을 **중간 두 줄을 가린 채** 완성해 본다. 코드 전체를 처음부터 외우기보다는 어떤 자리에 어떤 식이 들어가는지를 평가한다.
5. **실행값 확인**: 13장의 숫자와 결과가 어느 노트북·모델·k·표준화 상태에서 나왔는지 설명한다.
6. **최종 점검**: 18-1~18-4 체크박스가 전부 체크되고 100문제 철자·용법에 빈칸이 없어야 시험용 암기를 마쳤다고 볼 수 있다.

이 문서의 근거와 범위 경계

- **교수님 공지**: 시험 10/21, ~5장 선형회귀, 단답형(괄호·일부 구문·결과쓰기), 공통 모델 메서드/그래프/NumPy/`mean_absolute_error` 등.

- **교수님 정리 녹음**: 세 모델(KNN 분류·KNN 회귀·선형회귀), 훈련/테스트·평가·분류/회귀·표준점수, `n_neighbors`·`kneighbors`·`coef_`·`intercept_`, 코드 위주 출제, 패키지와 클래스 이름 암기 강조.
- **노트북 근거**: `01_KNN_.ipynb`, `02_ _ _.ipynb`, `03_KNN_.ipynb`, `04_KNN_.ipynb`, `05_ _ _.ipynb`.
- **자료 구분**: `stratify`, `np.vstack`, `np.full`, 그래프 일부 등은 **온라인 보충으로 추가된 노트북 셀**에서 근거를 얻었음. `np.mean`의 브로드캐스팅 설명, R^2 식·음수 가능성 등은 동작을 이해하기 위한 **해설**이며 별도로 교수님이 출제한다고 확정한 세부 문항은 아님.
- **의도적 제외**: 릿지·라쏘 등 5장 선형회귀 이후의 알고리즘, 과제 자료, 별도의 신규 라이브러리.
- **주의**: 실제 시험에서 만점을 보장할 수는 없음. 이 문서는 **제공된 수업 코드와 교수님이 알려준 출제 범위**를 가능한 상세히 외울 수 있게 만든 자료임.